Computer Architecture
CS 622 / EE 520
Spring 2026

Lecture 18

Shahid Masud

Further Readings

Reading Notes

Readings:
Appendix B of course textbook

Slide credits

Lecture 11: Memory Hierarchy
-- Cache Organization and Performance

CSCE 513 Computer Architecture

Department of Computer Science and Engineering
Yonghong Yan
yanyh@cse.sc.edu
https://passlab.github.io/CSCE513

Slides adapted from:
https://www.cs.nthu.edu.tw/~king/courses/cs5100/2014/N04-Cache-1.pdf

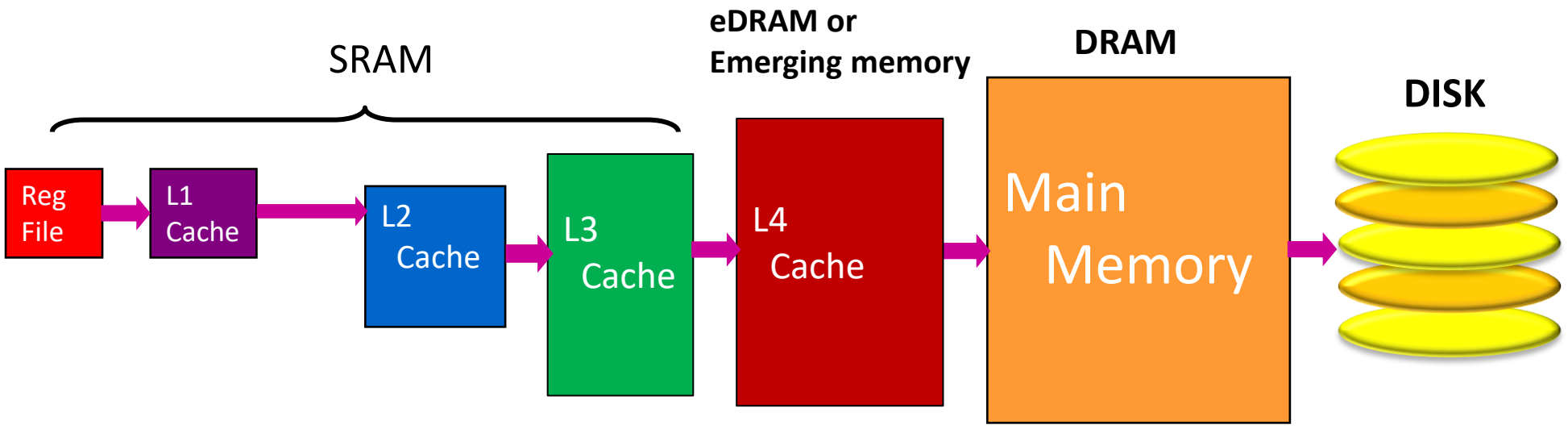
Topics

- Caches
  - Read/Write Mechanism
  - Organization
  - Replacement
  - Performance
  - Solved Questions

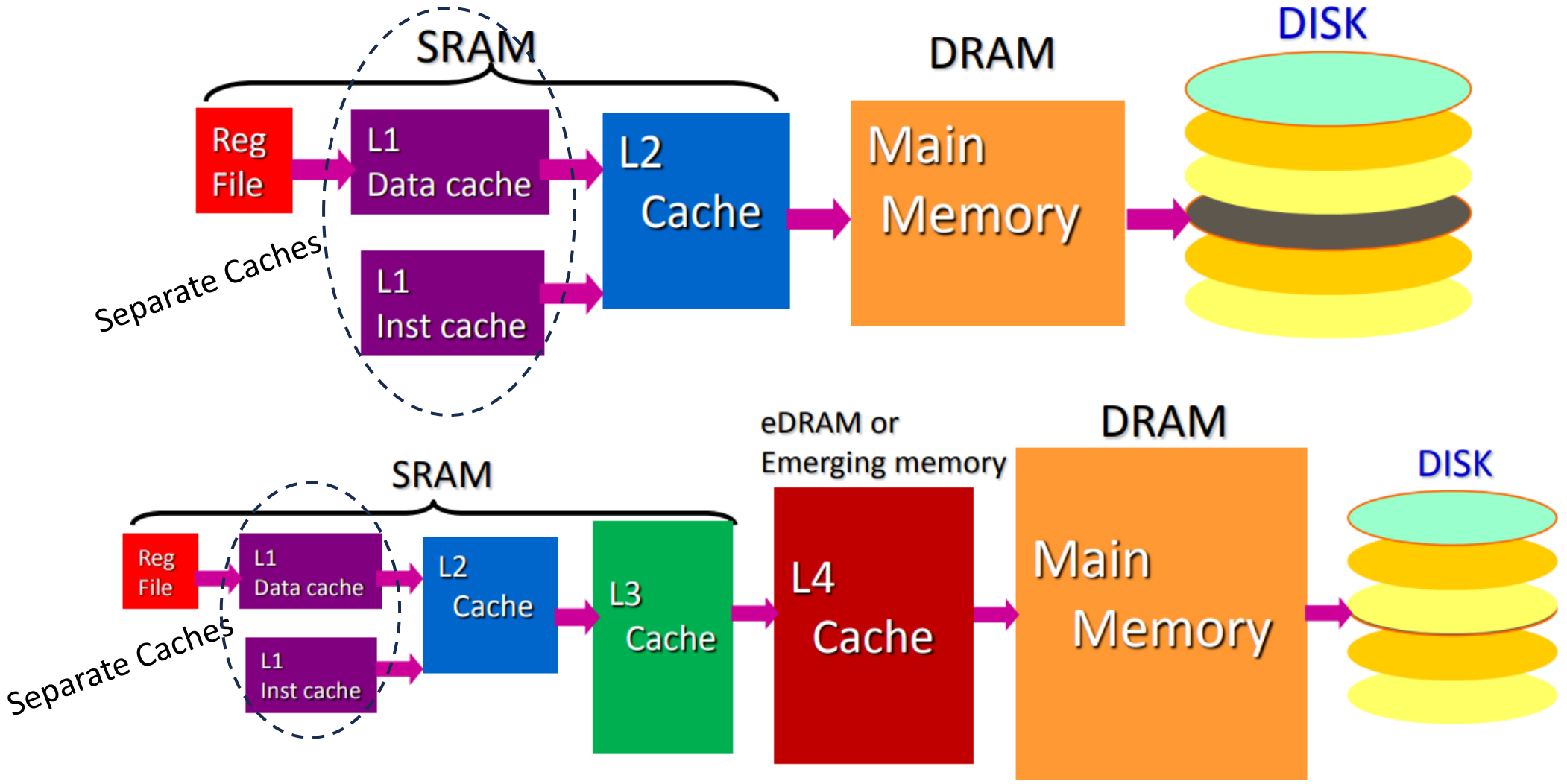
Cache Organization, Design and Access

Memory Hierarchy Design 1

- Entire addressable memory space available in largest, slowest memory
- Incrementally smaller and faster memories, each containing a subset of the memory below it, proceed in steps up toward the processor

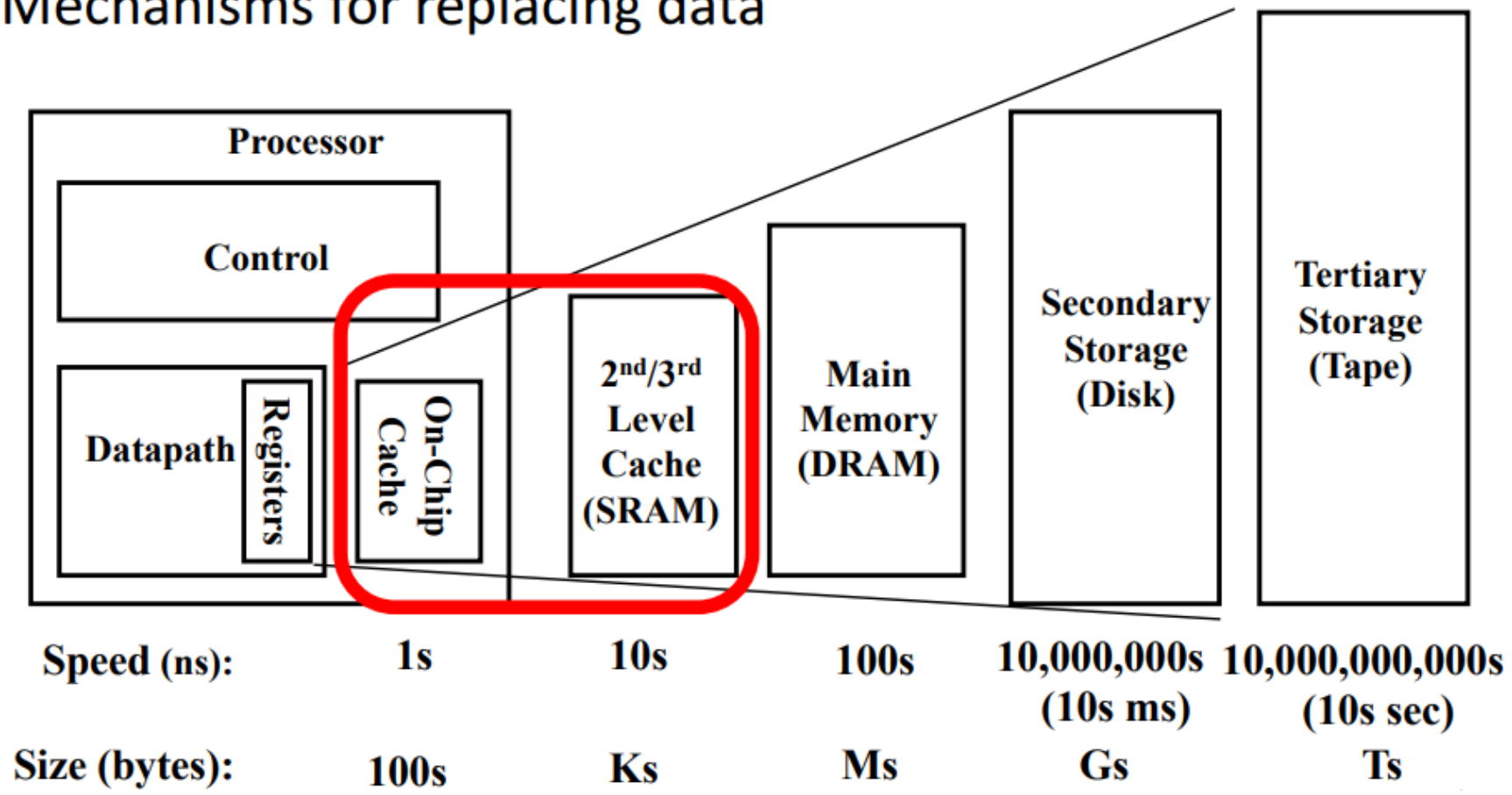


Another possible Memory Hierarchy



Memory Hierarchy Put to Use

- **Keep most recent accessed data and its adjacent data in the smaller/faster caches that are closer to processor**
- Mechanisms for replacing data



Cache and Memory Relation

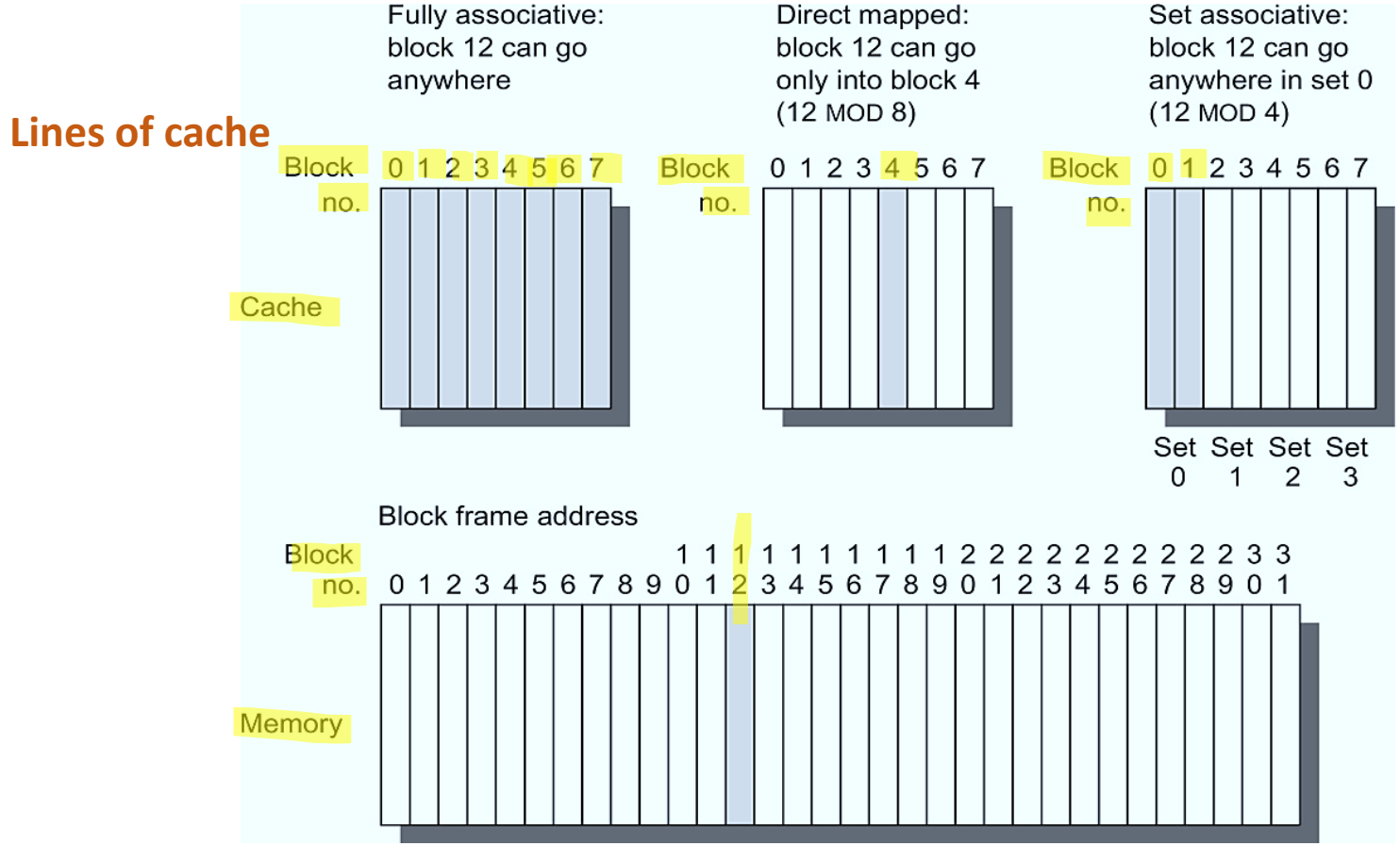
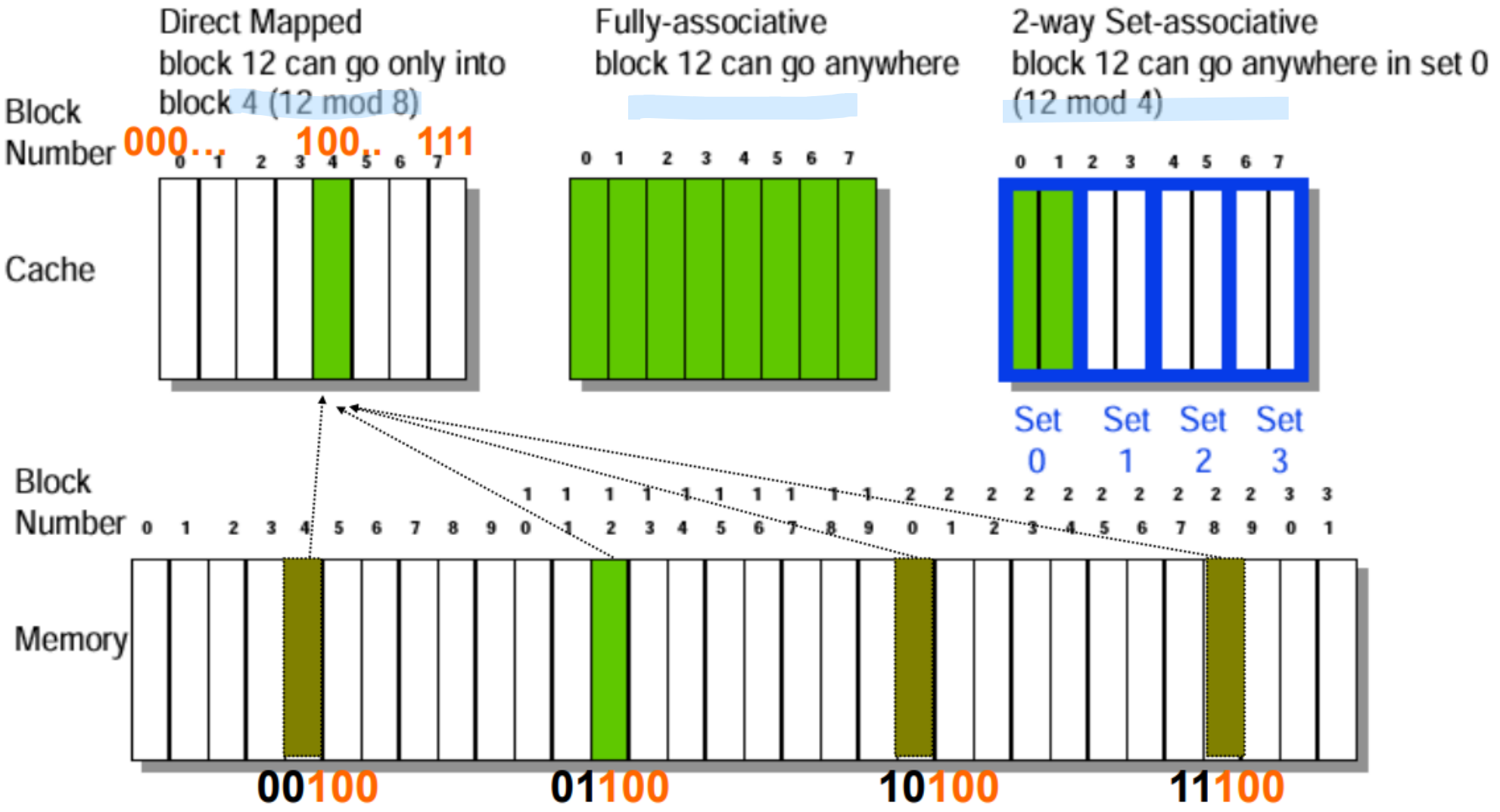


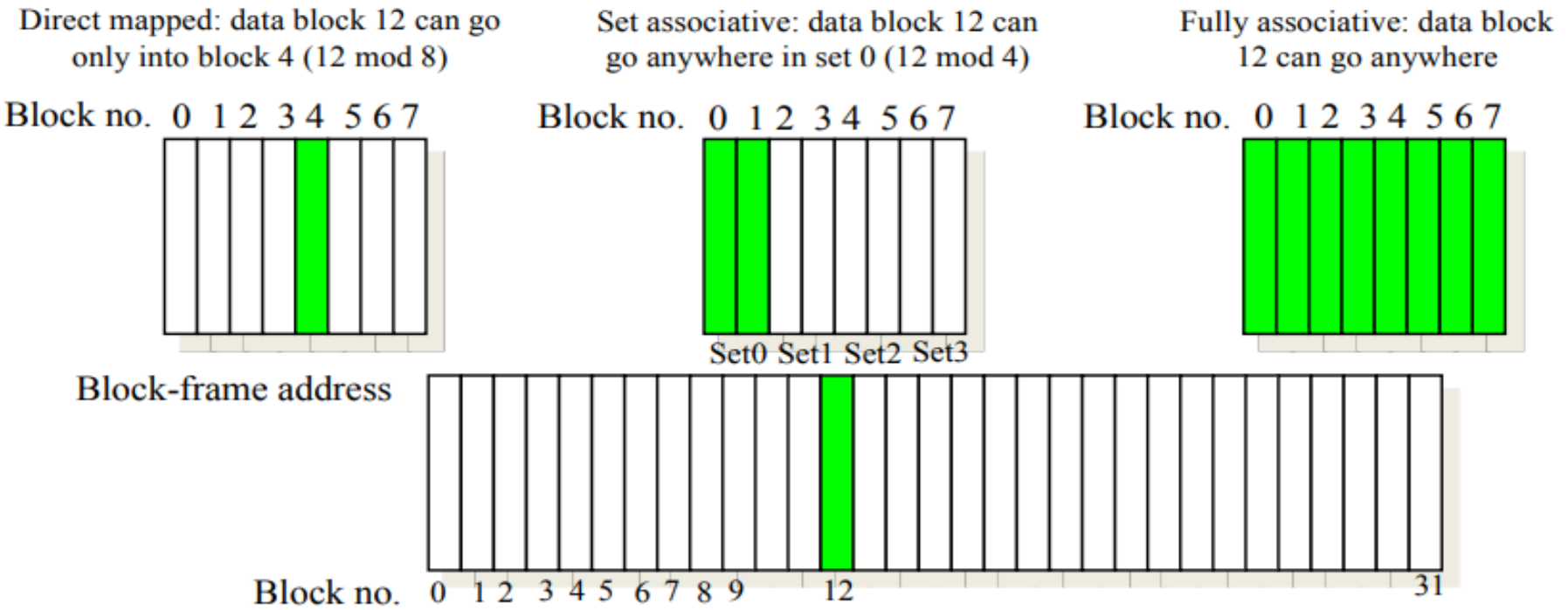
Figure B.2 This example cache has eight block frames and memory has 32 blocks. The three options for caches are shown left to right. In fully associative, block 12 from the lower level can go into any of the eight block frames of the cache. With direct mapped, block 12 can only be placed into block frame 4 (12 modulo 8). Set associative, which has some of both features, allows the block to be placed anywhere in set 0 (12 modulo 4). With two blocks per set, this means block 12 can be placed either in block 0 or in block 1 of the cache. Real caches contain thousands of block frames, and real memories contain millions of blocks. The set associative organization has four sets with two blocks per set, called *two-way set associative*. Assume that there is nothing in the cache and that the block address in question identifies lower-level block 12.

Cache Organization



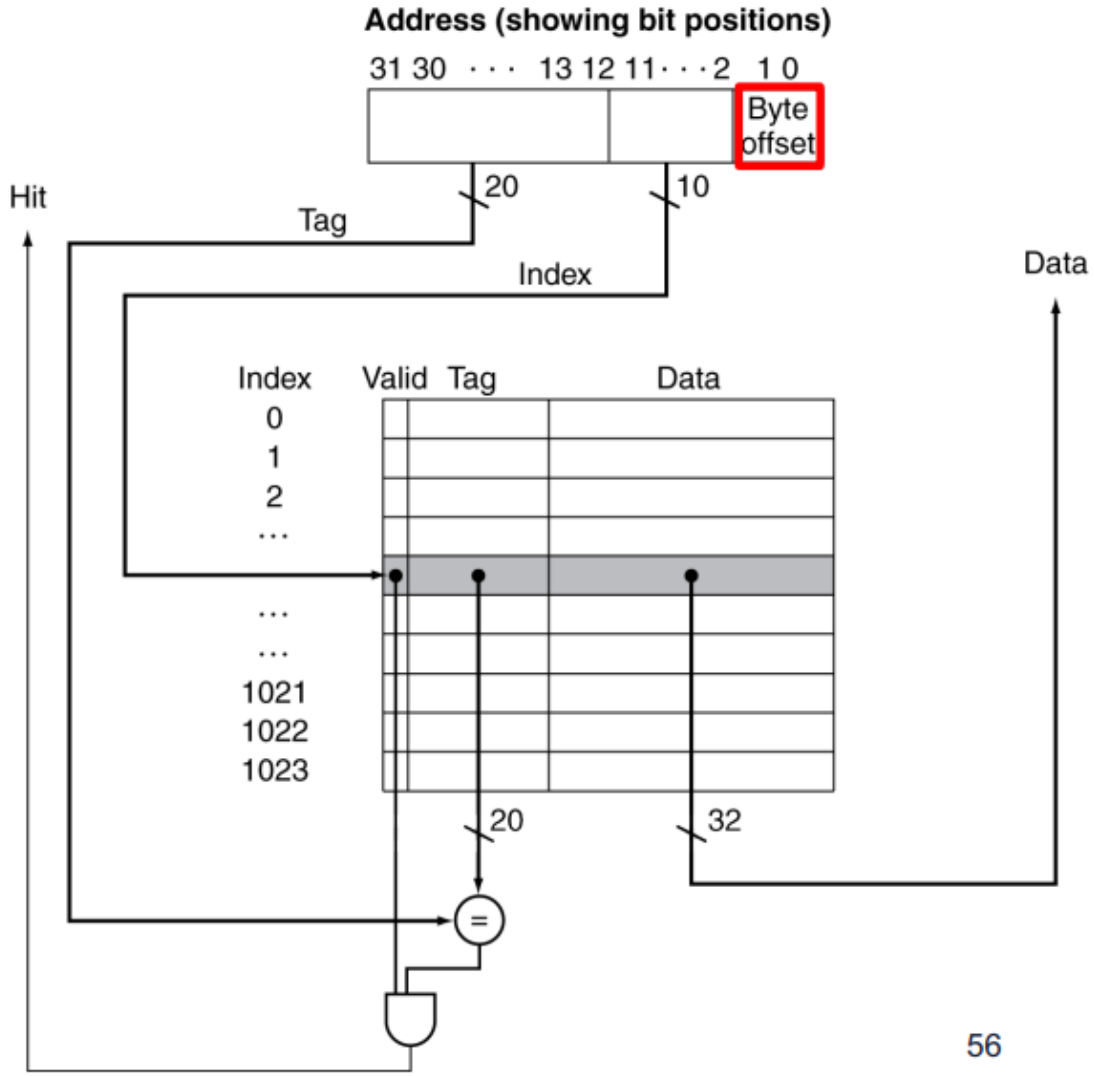
Where to place a Block (Line) of Data in Cache?

- Block Placement
  - Direct Mapped, Fully Associative, Set Associative
    - Direct mapped: (Block number) mod (Number of blocks in cache)
    - Set associative: (Block number) mod (Number of sets in cache)
      - # of set ≤ # of blocks
      - *n*-way: *n* blocks in a set
      - 1-way = direct mapped
    - Fully associative: # of set = 1



Cache Address Fields

- 4-byte data per cache entry: block or line size
  - Cache line
- **Why do we keep multiple bytes in one cache line?**
  - **Spatial locality**



Parts of Cache Address

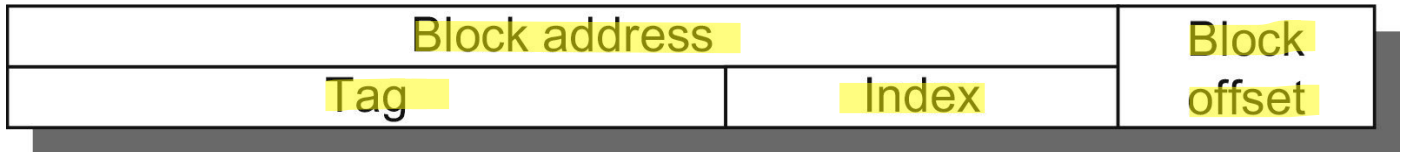


Figure B.3 The three portions of an address in a set associative or direct-mapped cache. The tag is used to check all the blocks in the set, and the index is used to select the set. The block offset is the address of the desired data within the block. Fully associative caches have no index field.

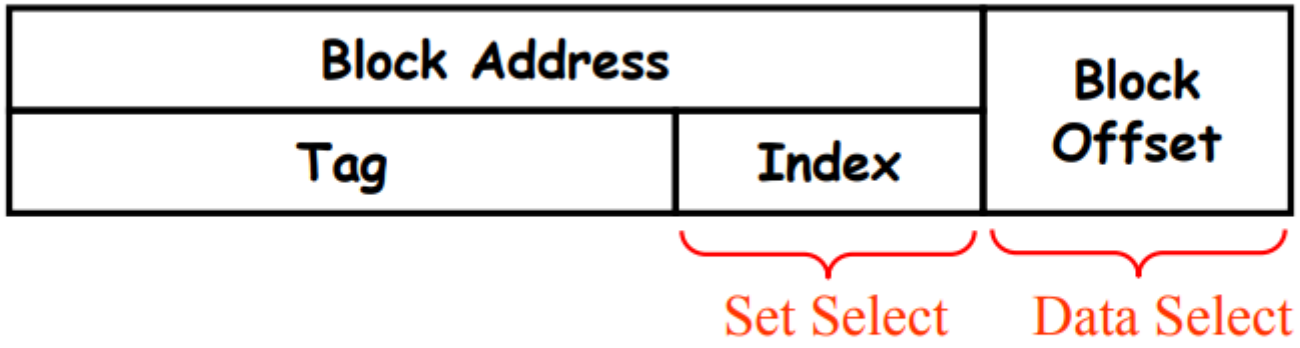
Identifying Data Location in a Cache

Cache Block

	Cache Line		

Q2: Block Identification

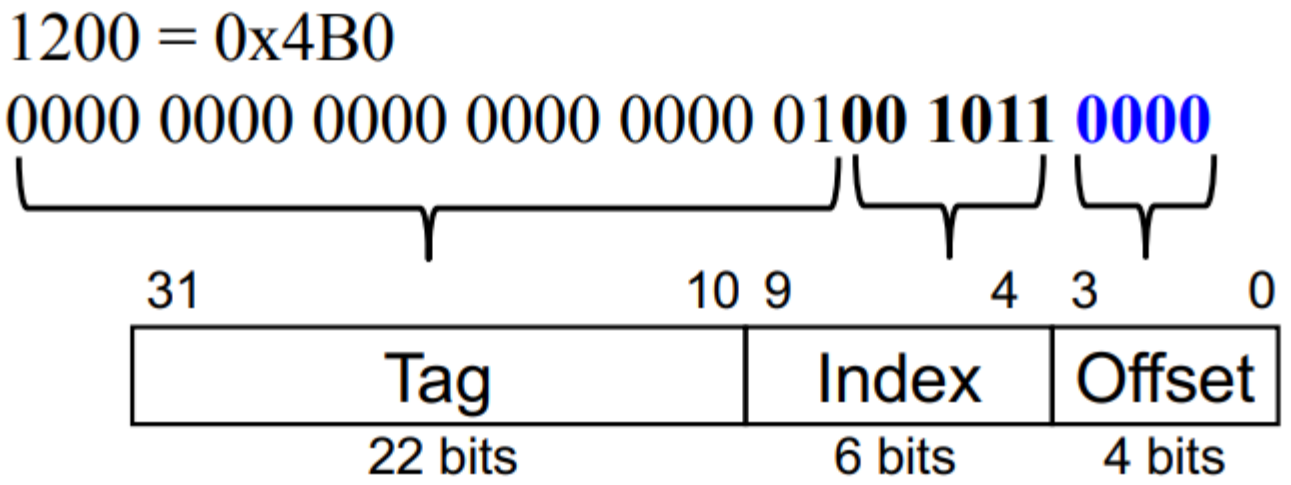
- Tag on each block
  - No need to check index or block offset
- Increasing associativity shrinks index, expands tag



$$\text{Cache size} = \text{Associativity} \times 2^{\text{index_size}} \times 2^{\text{offest_size}}$$

Example: Larger Block Size

- **64 blocks, 16 bytes/block**
  - To what block number does address 1200 (decimal) map?: 11

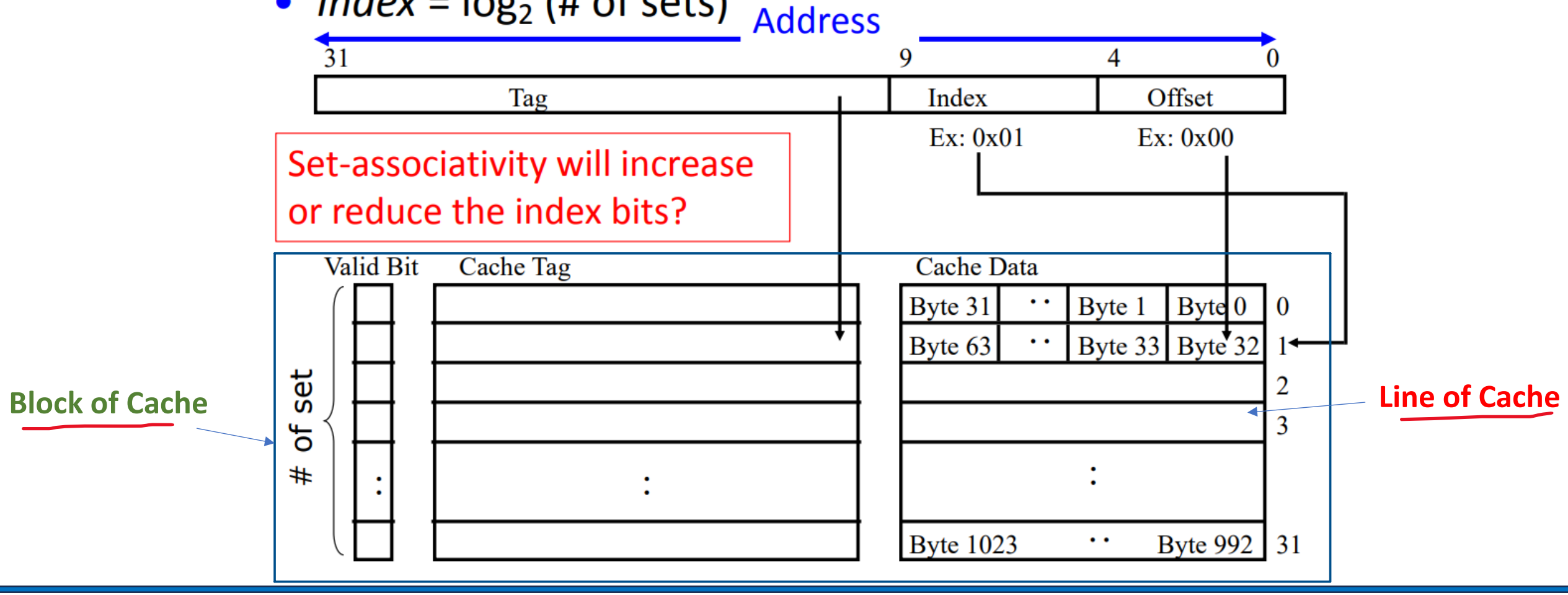


- Map all addresses between 1200 - 1215

Example of Direct Mapped DM Cache

Example: 1KB DM Cache, 32-byte Blocks

- The lowest M bits are the *offset* (block size = 2^M= 32)
- *Index* = $\log_2$ (# of sets)



Cache Example

- 8-blocks, 1 word/block, direct mapped
- Initial state

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

Cache Example 1

Word addr	Binary addr	Hit/miss	Cache block
26	**11 010**	Miss	010

Index	V	Tag	Data
000	N		
001	N		
010	**Y**	**11**	**Mem[11010]**
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example 1

Word addr	Binary addr	Hit/miss	Cache block
22	**10** 110	Hit	110
26	**11** 010	Hit	010

Index	V	Tag	Data
000	N		
001	N		
010	Y	11	Mem[11010]
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example 2

Word addr	Binary addr	Hit/miss	Cache block
16	**10 000**	Miss	000
3	**00 011**	Miss	011
16	**10 000**	Hit	000

Index	V	Tag	Data
000	**Y**	**10**	**Mem[10000]**
001	N		
010	Y	11	Mem[11010]
011	**Y**	**00**	**Mem[00011]**
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Organization of Cache in Opteron Processor

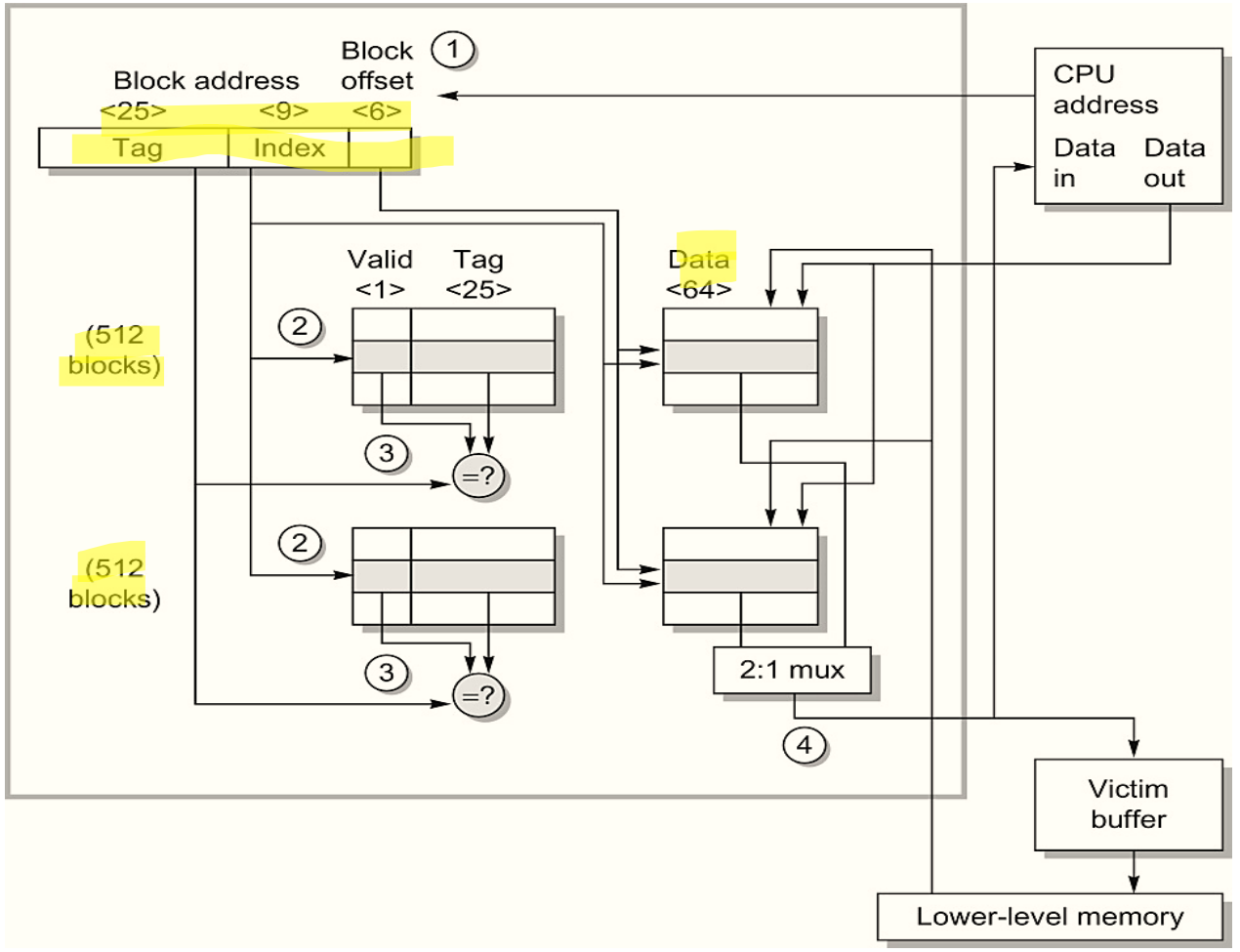
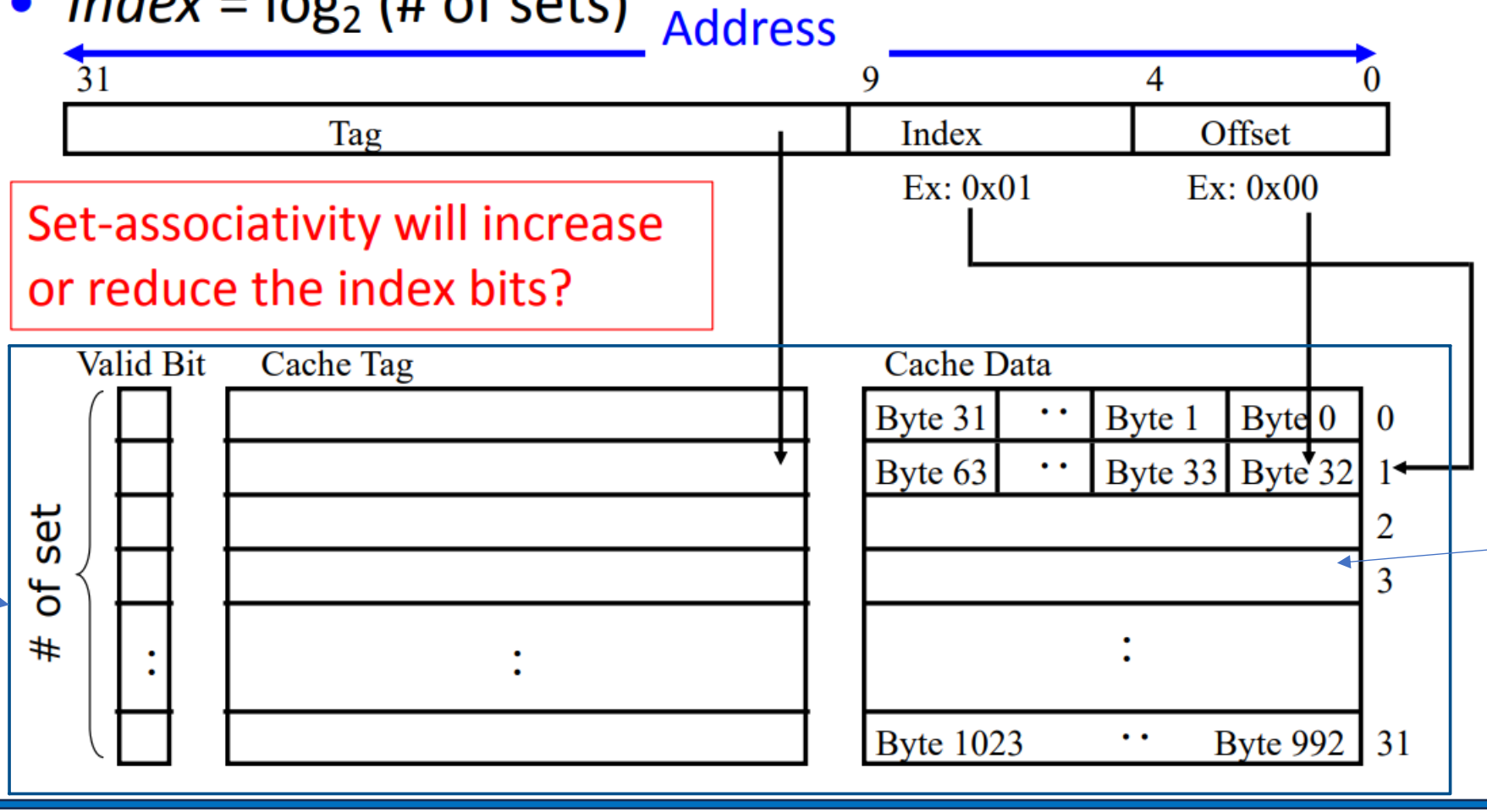


Figure B.5 The organization of the data cache in the Opteron microprocessor. The 64 KiB cache is two-way set associative with 64-byte blocks. The 9-bit index selects among 512 sets. The four steps of a read hit, shown as circled numbers in order of occurrence, label this organization. Three bits of the block offset join the index to supply the RAM address to select the proper 8 bytes. Thus, the cache holds two groups of 4096 64-bit words, with each group containing half of the 512 sets.

Example of Direct Mapped DM Cache

Example: 1KB DM Cache, 32-byte Blocks

- The lowest M bits are the *offset* (block size = 2^M= 32)
- *Index* = $\log_2$ (# of sets)

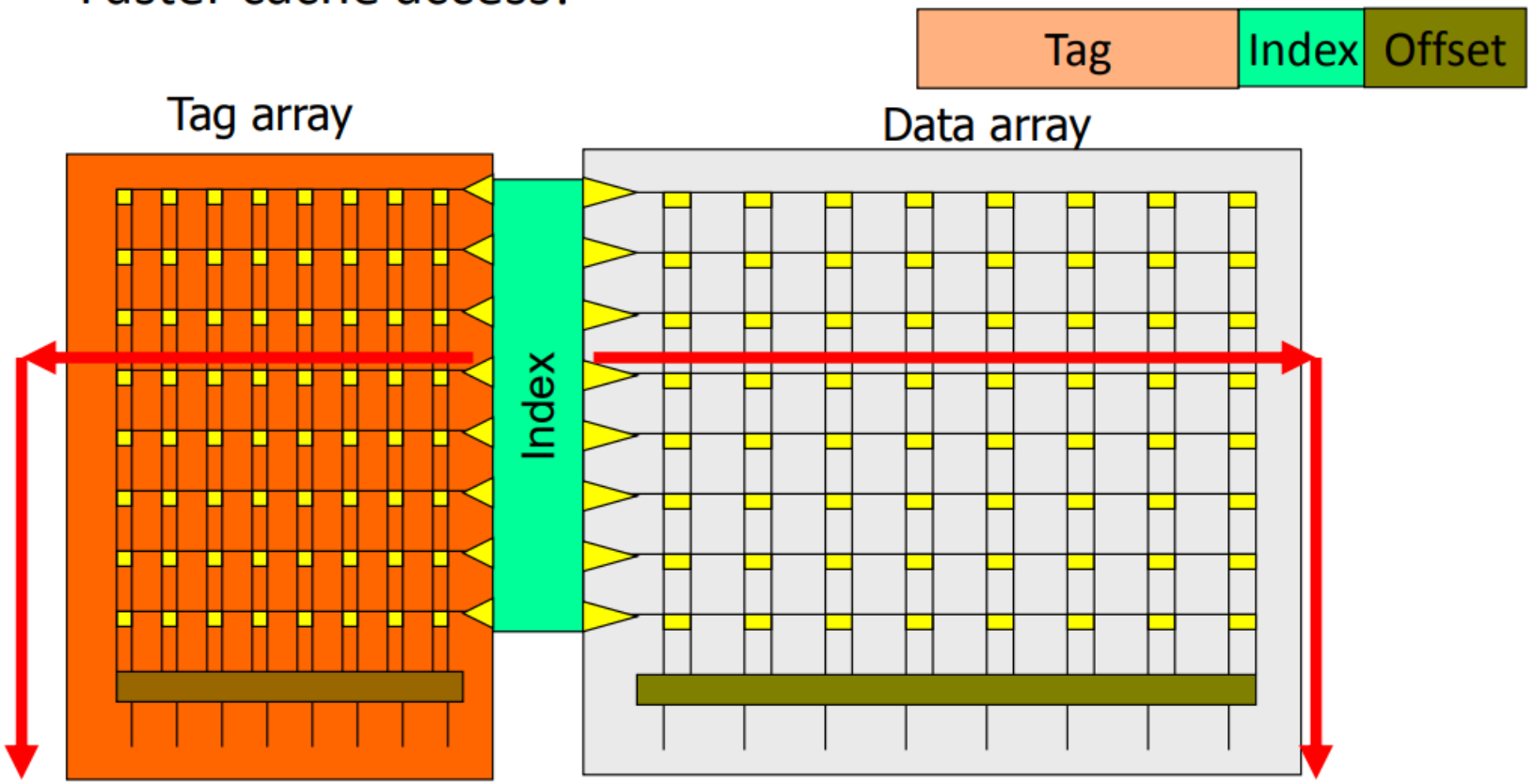


Comparison of Cache Architecture

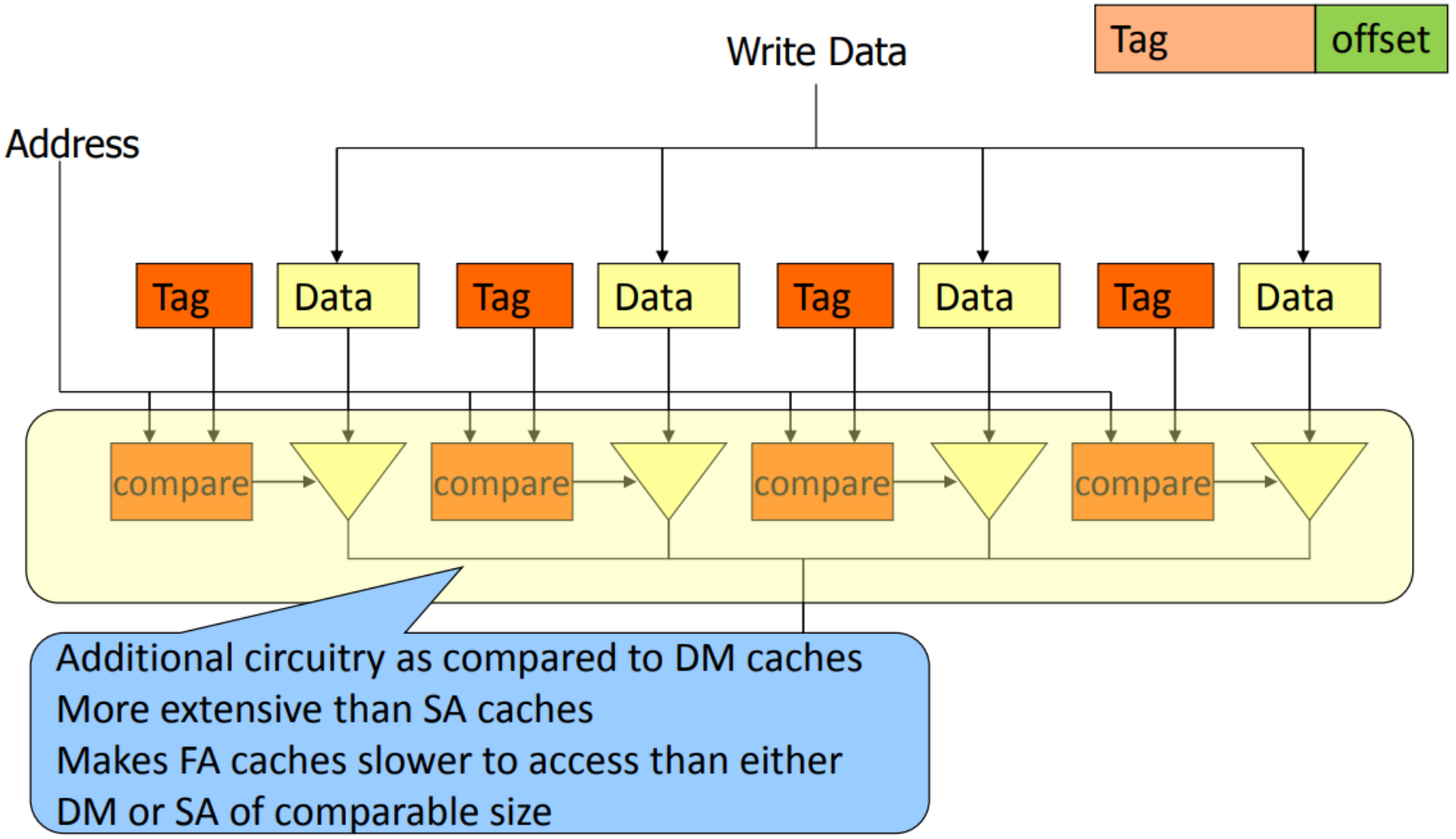
- Comparisons in terms of space and time:
- Can access to tag and data arrays be done in parallel?
- How many comparators and multiplexers are needed?
  - How about wiring in IC layout?
- How many bits of storage are needed in tag and data arrays?
  - The index bits need not be stored!
- What is the write overhead?

Fast – Direct Mapped Cache

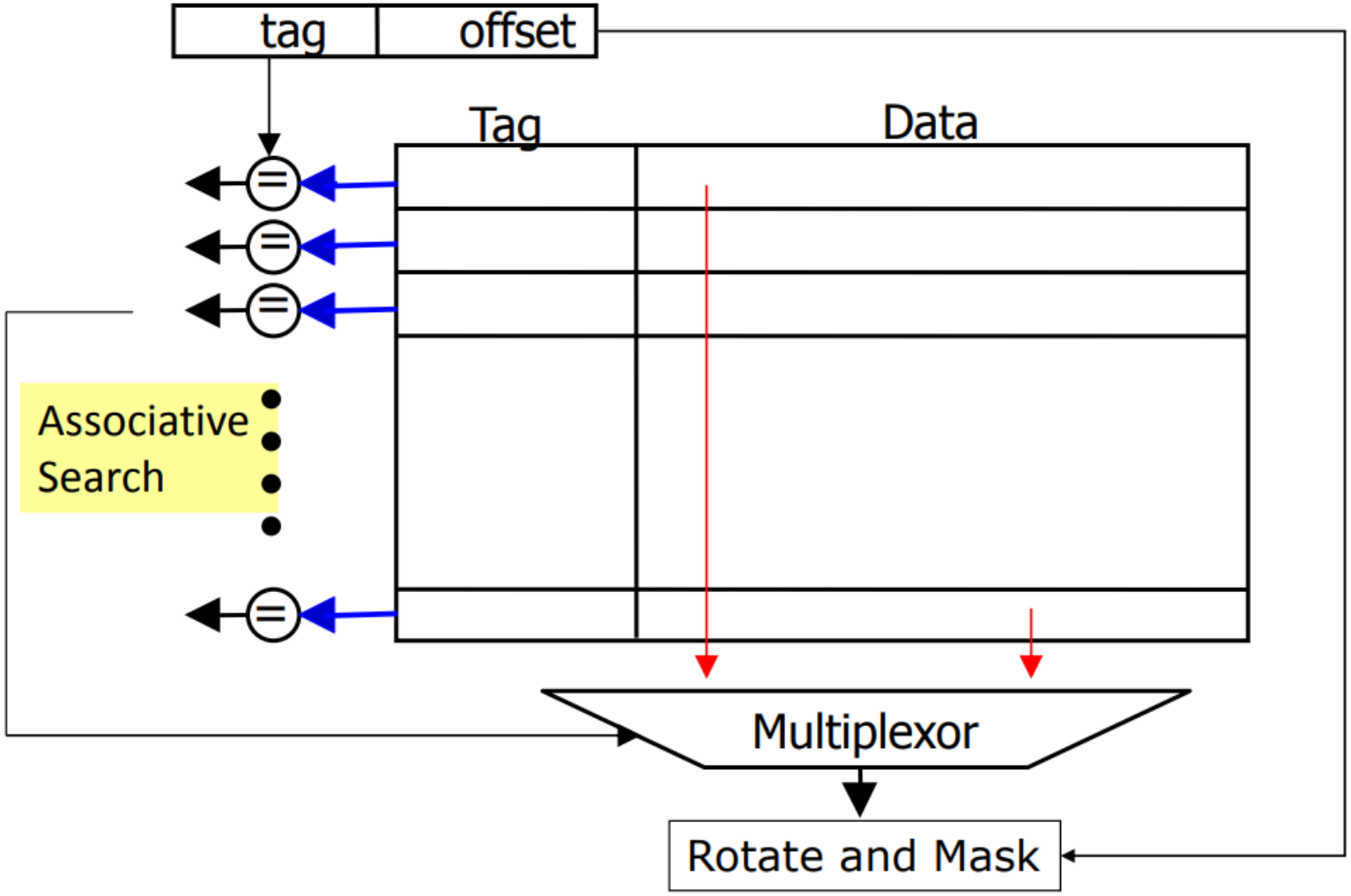
- Tag and data access happen in parallel
  - Faster cache access!



Overhead of Fully Associative Cache – 1

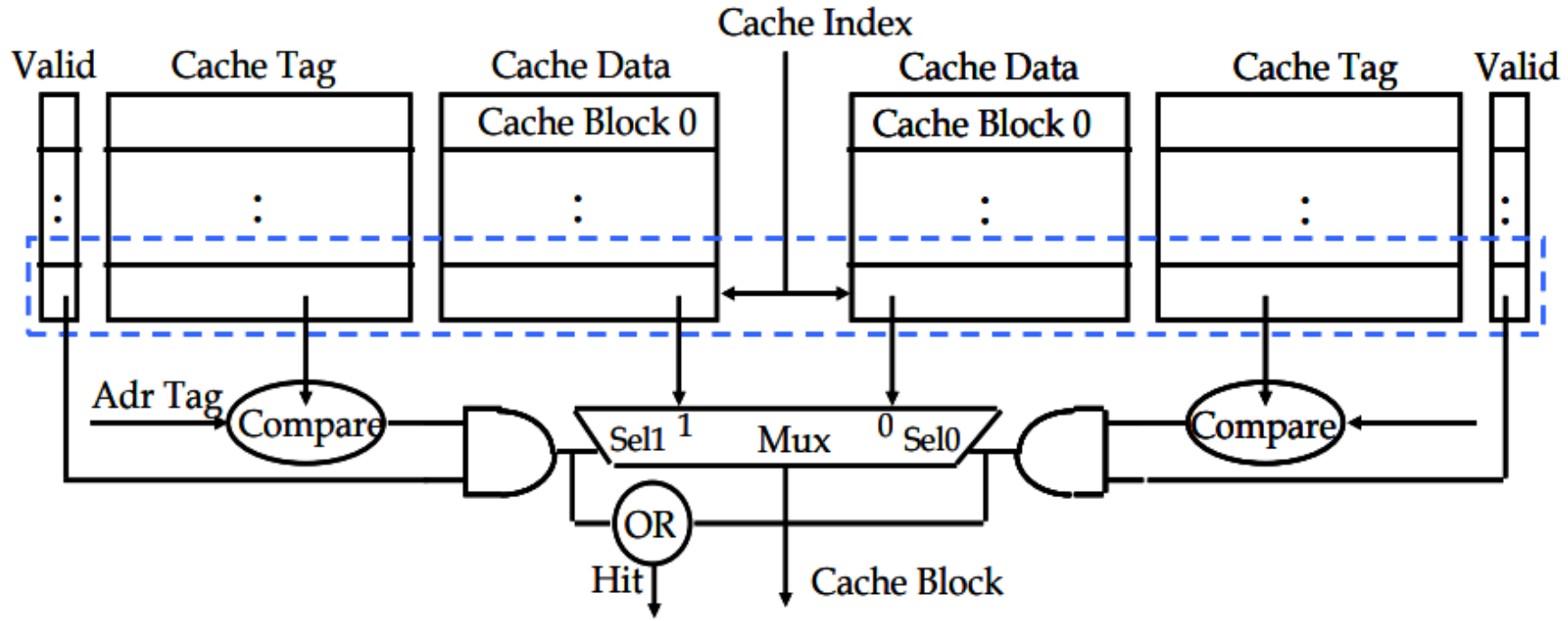


Overhead of Full Associative Cache – 2

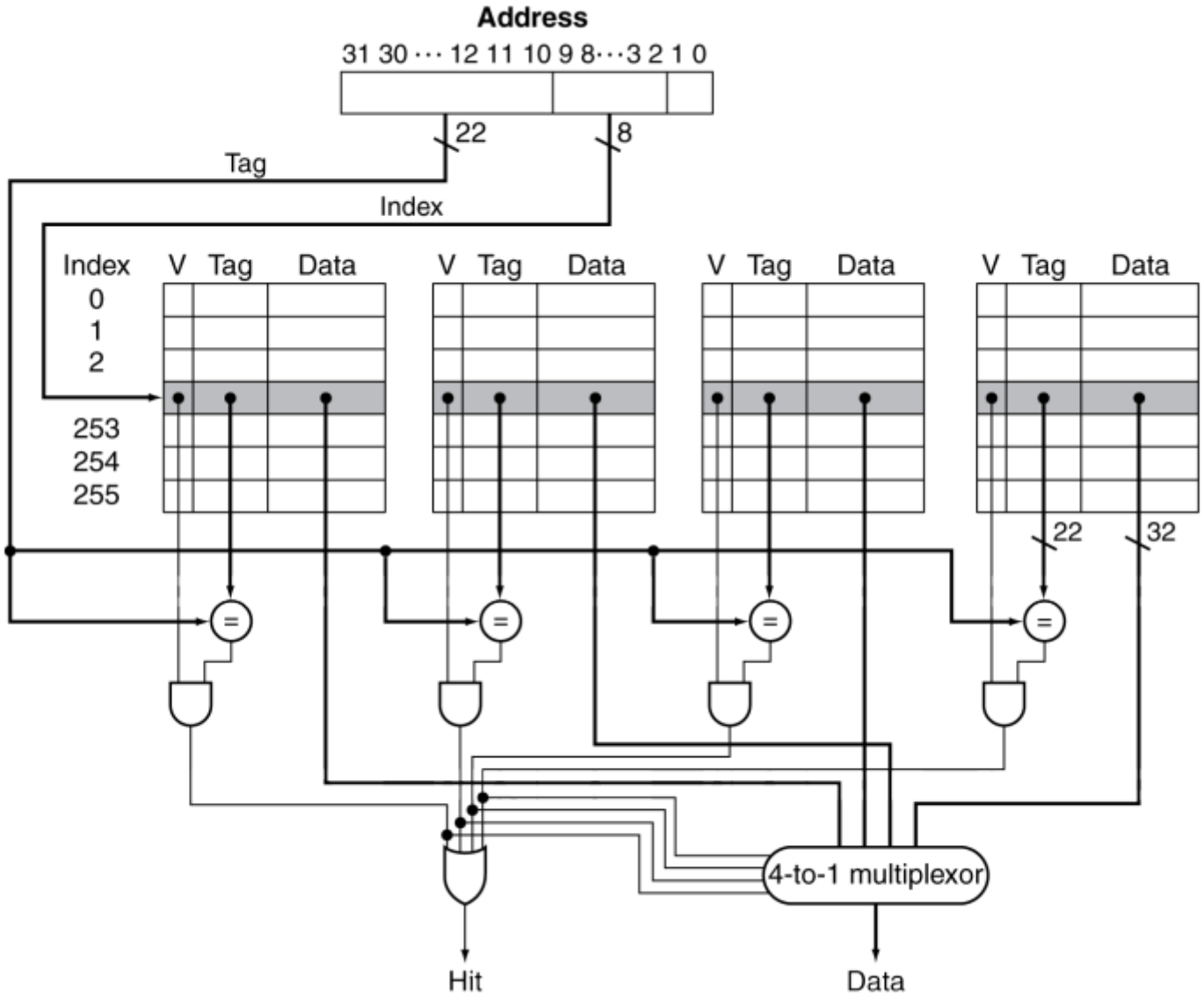


Architecture of Set Associative Cache - 1

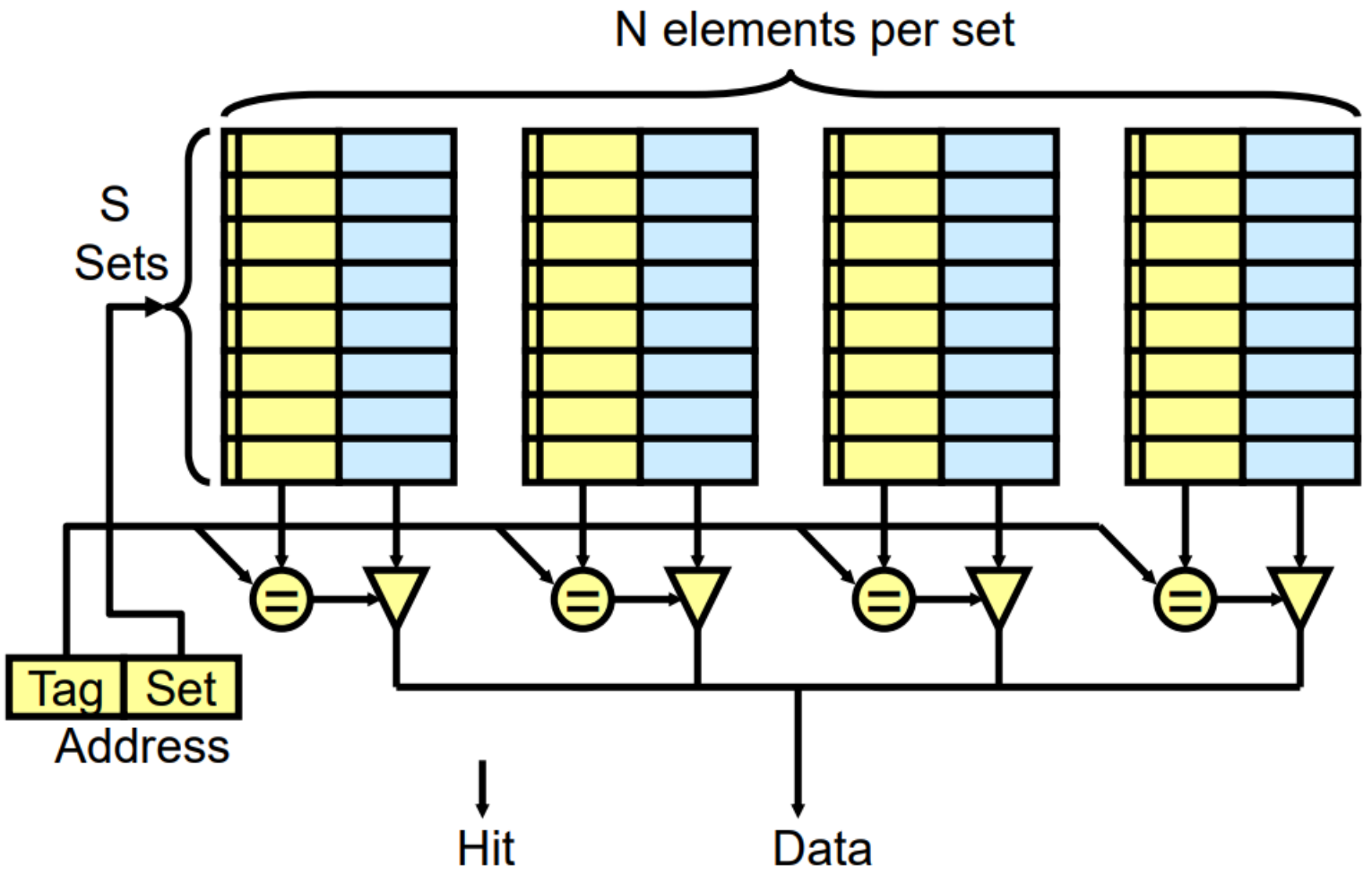
- *N*-way set associative: *N* entries for each **Cache Index**
  - *N* direct mapped caches operates in parallel
- Example: Two-way set associative cache
  - **Cache Index** selects a "set" from the cache;
  - The two tags in the set are compared to the input in parallel;
  - Data is selected based on the tag result.



Architecture of Set Associative Cache - 2



Set Associative Cache



- Share costly comparators across sets; MUX delay

Examples of Associativity in Cache

- 2-way set associative

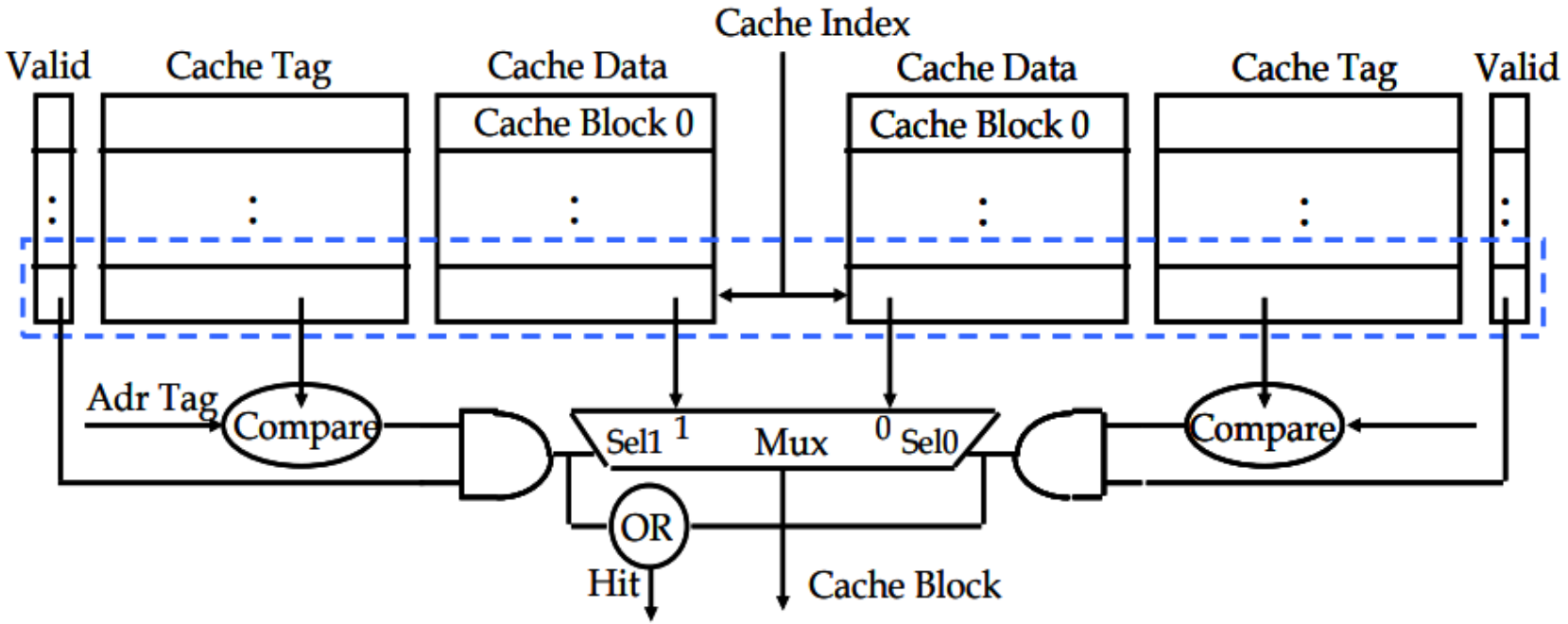
Block address	Cache index	Hit/miss	Cache content after access			
			Set 0		Set 1	
0	0	miss	**Mem[0]**			
8	0	miss	Mem[0]	**Mem[8]**		
0	0	hit	**Mem[0]**	Mem[8]		
6	0	miss	Mem[0]	**Mem[6]**		
8	0	miss	**Mem[8]**	Mem[6]		

- Fully associative

Block address		Hit/miss	Cache content after access			
0		miss	**Mem[0]**			
8		miss	Mem[0]	**Mem[8]**		
0		hit	**Mem[0]**	Mem[8]		
6		miss	Mem[0]	Mem[8]	**Mem[6]**	
8		hit	Mem[0]	**Mem[8]**	Mem[6]	

Disadvantage of Set Associative Cache

- *N*-way Set Associative Cache versus Direct Mapped Cache:
  - N comparators vs. 1
  - Extra MUX delay for the data
  - Data comes AFTER Hit/Miss decision and set selection
- In a direct mapped cache, Cache Block is available BEFORE Hit/Miss:
  - Possible to assume a hit and continue. Recover later if miss.



Questions on Cache Organization

- Which cache organization can reduce compulsory misses?
- Which cache organization can reduce capacity misses?
- Which cache organization can reduce conflict misses?
- Reduce cache misses = improve cache performance?

Cache Hit and Miss

What is Cache Hit and Miss?

To study the Memory Hierarchy behaviour when a Read or Write Command finds:

that Data is available inside a Cache

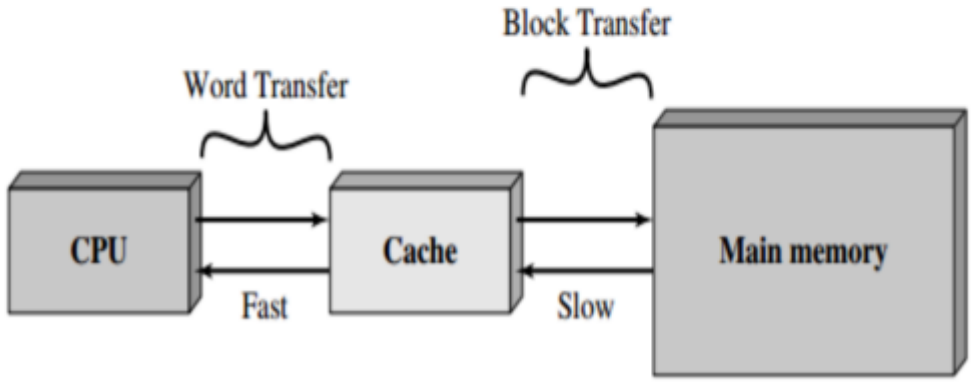
Or

When the Data is Not Available inside a Cache

Considerations for Cache Design

- Q1: Where can a block be placed in the upper level?
  - Block placement
- Q2: How is a block found if it is in the upper level?
  - Block identification
- Q3: Which block should be replaced on a miss?
  - Block replacement
- Q4: What happens on a write?
  - Write strategy

Read hit



(a) Single cache

Studying Replacement Policies

	Associativity								
	Two-way			Four-way			Eight-way		
Size	LRU	Random	FIFO	LRU	Random	FIFO	LRU	Random	FIFO
16 KiB	114.1	117.3	115.5	111.7	115.1	113.3	109.0	111.8	110.4
64 KiB	103.4	104.3	103.9	102.4	102.3	103.1	99.7	100.5	100.3
256 KiB	92.2	92.1	92.5	92.1	92.1	92.5	92.1	92.1	92.5

Figure B.4 Data cache misses per 1000 instructions comparing least recently used, random, and first in, first out replacement for several sizes and associativities. There is little difference between LRU and random for the largest size cache, with LRU outperforming the others for smaller caches. FIFO generally outperforms random in the smaller cache sizes. These data were collected for a block size of 64 bytes for the Alpha architecture using 10 SPEC2000 benchmarks. Five are from SPECint2000 (gap, gcc, gzip, mcf, and perl) and five are from SPECfp2000 (applu, art, equake, lucas, and swim). We will use this computer and these benchmarks in most figures in this appendix.

Study Cache Miss Behavior

Size (KiB)	Instruction cache	Data cache	Unified cache
8	8.16	44.0	63.0
16	3.82	40.9	51.0
32	1.36	38.4	43.3
64	0.61	36.9	39.4
128	0.30	35.3	36.2
256	0.02	32.6	32.9

Figure B.6 Miss per 1000 instructions for instruction, data, and unified caches of different sizes. The percentage of instruction references is about 74%. The data are for two-way associative caches with 64-byte blocks for the same computer and benchmarks as Figure B.4.

What is Hit and Miss?

To study the Memory Hierarchy behaviour when a Read or Write Command discovers that the Data is available inside a Cache

Write Hit Policy

- Write through
  - Update next level on every write → cache is always clean
  - Lots of traffic to next level (mostly writes)
- Write back
  - Write to cache and mark block dirty → update main memory on eviction
  - More complex eviction and coherence
- Reservation problem:
  - Reads use tag and data array at same time
  - Writes use tag first, then data array
  - How to pipeline to allow one read/write per cycle?

Sequence of Updation?

What Happens on a Write Command

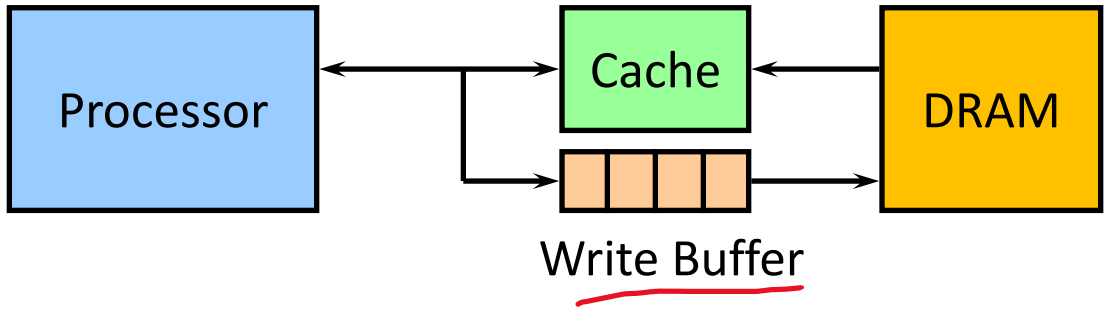
Q4: What Happens on a Write?

	Write-Through	Write-Back
Policy	Data written to cache block, also written to lower-level memory	1. Write data only to the cache 2. Update lower level when a block falls out of the cache
Debug	Easy	Hard
Do read misses produce writes?	No	Yes
Do repeated writes make it to lower level?	Yes	No

Additional option -- let writes to an un-cached address allocate a new cache line ("write-allocate").

Write Hit Approach

- Write through
  - Update next level on every write → cache is always clean
  - Lots of traffic to next level (mostly writes)
- Write back
  - Write to cache and mark block dirty → update main memory on eviction (more complex eviction and coherence)
- Write buffer for write through
  - Processor: writes data into cache and write buffer
  - *Memory controller*: write contents of buffer to memory



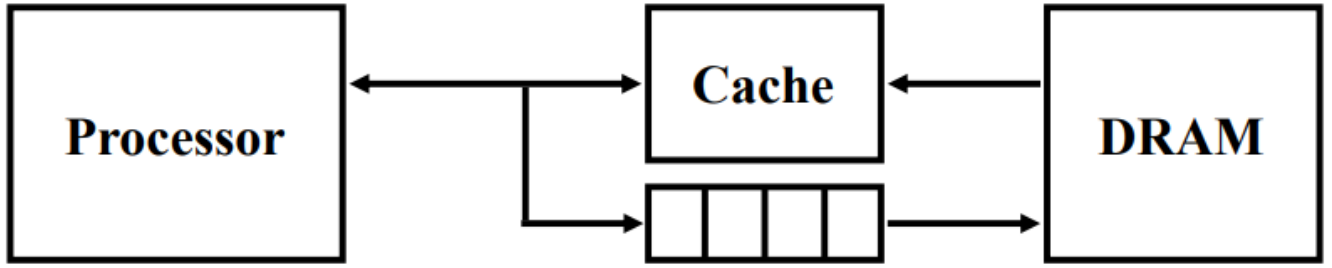
What Happens on a Write Command

	Write-Through	Write-Back
Policy	Data written to cache block, also written to lower-level memory	1. Write data only to the cache 2. Update lower level when a block falls out of the cache
Debug	Easy	Hard
Do read misses produce writes?	No	Yes
Do repeated writes make it to lower level?	Yes	No

Additional option -- let writes to an un-cached address allocate a new cache line ("write-allocate").

Write Stall

When the processor must wait for writes to complete during write through, the processor is said to *write stall*. A common optimization to reduce write stalls is a *write buffer*, which allows the processor to continue as soon as the data are written to the buffer, thereby overlapping processor execution with memory updating. As we will see shortly, write stalls can occur even with write buffers.



- Q. Why a write buffer ?
  - A. So CPU doesn't stall
- Q. Why a buffer, why not just one register ?
  - A. Bursts of writes are common.
- Q. Are Read After Write (RAW) hazards an issue for write buffer?
  - A. Yes! Drain buffer before next read, or send read 1st after check write buffers.

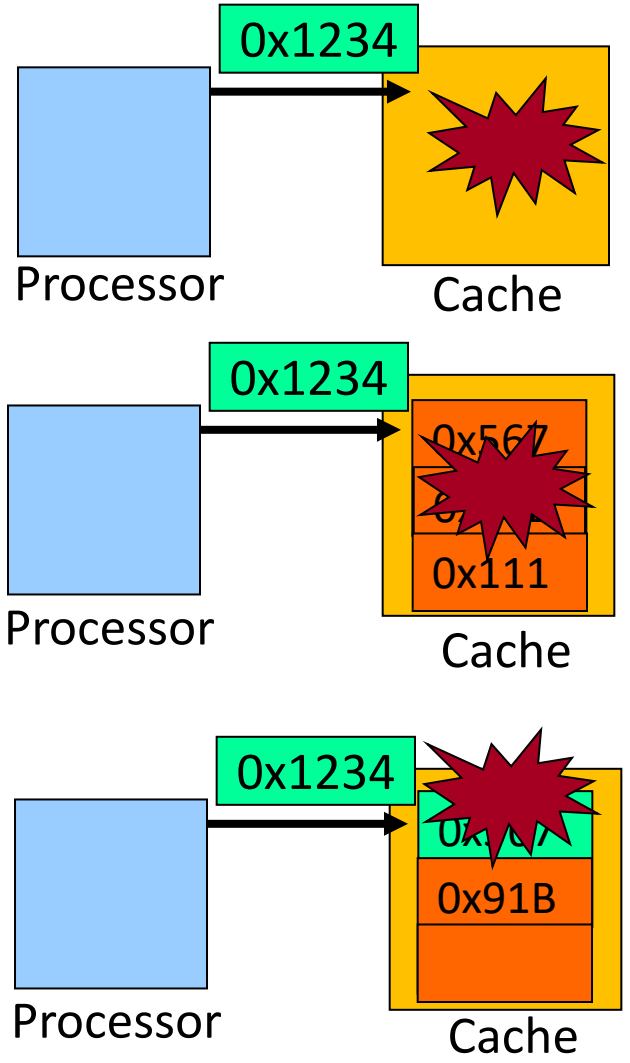
Cache Write Miss, Cache Replacement

Write Miss Policy

- **Write allocate**:
  - Allocate a new block on each write
  - Fetch on write: fetch entire block, then write word into block
  - No-fetch: allocate block but don't fetch
    - Requires valid bits per word, complex eviction
- **Write no-allocate**:
  - Write around cache; typically used by write through
- **Write invalidate** (instead of update)
- Sometimes like to have a read no-allocate
  - irregular accesses on machine with large block size

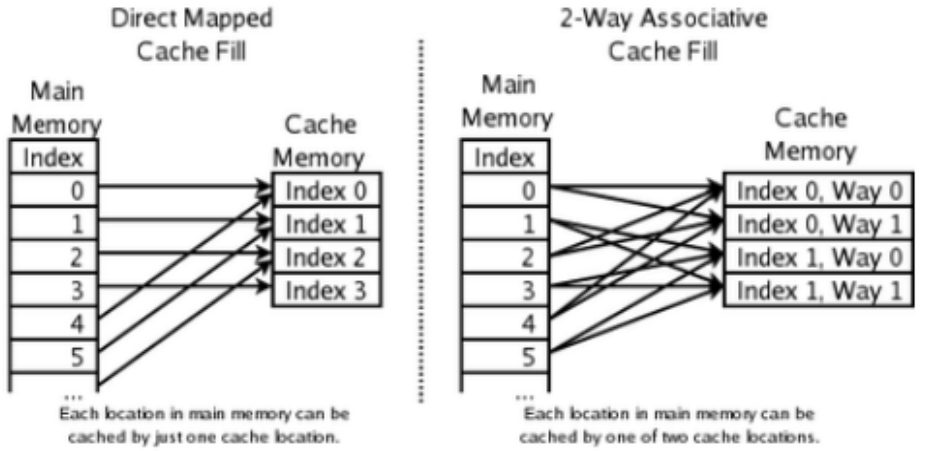
Types of Cache Miss

- Cache miss as a metric: (causes of cache misses)
  - **Compulsory**:
    - First access to a block
  - **Capacity**:
    - Block discarded due to limited cache size and later retrieved
  - **Conflict**:
    - Block discarded due to conflict in set and later retrieved
  - **Coherence**:
    - In multicore systems (Chapter 5)



Sources of Cache Misses

- **Compulsory** (cold start or process migration, first reference): first access to a block
  - "Cold" fact of life: not a whole lot you can do about it
  - **Note: If you are going to run "billions" of instruction, Compulsory Misses are insignificant**
- **Conflict** (collision):
  - Multiple memory locations mapped to the same cache location
  - **Solution 1: increase cache size**
  - **Solution 2: increase associativity**



- **Capacity**:
  - Cache cannot contain all blocks access by the program
  - **Solution: increase cache size**

- Coherence (Invalidation): other process (e.g., I/O) updates memory
  - We will cover it later on.

Example of Write Miss Policy

- Example: Assume a fully associative write-back cache with many cache entries that starts empty. Below is sequence of five memory operations (The address is in square brackets):

  Write Mem[100];
  Write Mem[100];
  Read Mem[200];
  Write Mem[200];
  Write Mem[100].

  What are the number of hits and misses (inclusive reads and writes) when using no-write allocate versus write allocate?

- *Answer*

No-write Allocate:

Write Mem[100]; 1 write miss
Write Mem[100]; 1 write miss
Read Mem[200]; 1 read miss
Write Mem[200]; 1 write hit
Write Mem[100]. 1 write miss
4 misses; 1 hit

Write allocate:

Write Mem[100]; 1 write miss
Write Mem[100]; 1 write hit
Read Mem[200]; 1 read miss
Write Mem[200]; 1 write hit
Write Mem[100]; 1 write hit
2 misses; 3 hits

Cache Replacement Policy

- On a cache miss we need to evict a block to make room for the new line
- In an n-way set associative cache, we have n choices of which block to evict
- Which block gets evicted?
- Optima cache replacement policy: [Belady, IBM Systems Journal, 1966]
  - Evict block with longest reuse distance
    - i.e. next reference to block is farthest in future
    - Requires knowledge of the future!
  - Can't build it, but can approximate it

Replacement Policy: NRU, Others

- For 2-way set-associative cache
  - 1-bit per set indicates LRU/MRU; set/clear on each access
- For >2-way, LRU is difficult/expensive in HW
  - Need to maintain history of access order
- Not recently used (NRU)
  - A "*used bit*" is associated with every block, initially 0
  - Whenever a block is accessed, its used bit is set to 1
    - If on an access, all other used bits in a set are 1, they are reset to 0 except the bit of the block that is accessed
  - A *replacement pointer* is used to look for a block whose used bit is 0 to be replaced
  - Used by Intel Itanium, Sparc T2
- Other policies: Pseudo LRU, LRU Insertion Policy (LIP)

Questions for Cache Hit and Miss

- ***Block placement***: where in upper hierarchy level?
  - fully associative, direct mapped, set-associative
- ***Block identification***: find block in upper hierarchy level?
  - search and match address tag:
    block address (= tag + index) + block offset
  - valid bit: tag-index boundary and associativity
- ***Block replacement***: which block to replace on miss?
  - easy for direct map; random/LRU for associative
- ***Write strategy***: what happens on a write?
  - Write through and write back
  - Write allocate and not allocate

Cache and CPU Performance

Memory Hierarchy Performance - AMAT

- **Average memory access time between levels**
  = Hit time + Miss rate x Miss penalty
- **Miss penalty**: time to fetch a block from lower memory level
  - **Access time**: time to lower level; function of latency
  - **Transfer time**: function of bandwidth between levels
    - Transfer one “cache line/block” at a time
    - Transfer at the size of the memory-bus width

Cache and CPU Performance

- **CPU time** = (CPU cycle + memory stall cycles) x clock cycle time
- **Memory stall cycles** = memory access x miss rate x miss penalty
  - Assumptions: memory stalls are due to cache misses, hit cycles included in CPU execution cycles, reads and writes have same miss rate and penalty
- **CPU time** = instruction count x (CPI_{ideal} + memory access/instruction x miss rate x miss penalty) x clock cycle time
- Cache design becomes more important for CPUs with lower CPI and higher clock rates

Average Memory Access Time

- Average Memory Access Time (AMAT)
  = Hit Time + Miss rate * Miss Penalty
  = T_{hit}(L1) + Miss%(L1) * T(memory)
- Example:
  - Cache Hit = 1 cycle
  - Miss rate = 10% = 0.1
  - Miss penalty = 300 cycles
  - AMAT = 1 + 0.1 * 300 = 31 cycles

